

第52章 MPU6050 传感器—姿态检测

本章参考数据：《STM32F4xx 参考手册》、《STM32F4xx 规格书》、库说明文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

关于 MPU6050 的参考资料：《MPU-60X0 寄存器》、《MPU6050 数据手册》以及官方驱动《motion_driver_6.12》。

本章讲解的内容跨领域的知识较多，若您感兴趣，请自行查阅各方面的资料，对比学习。

52.1 姿态检测

1. 基本认识

在飞行器中，飞行姿态是非常重要的参数，见图 52-1，以飞机自身的中心建立坐标系，当飞机绕坐标轴旋转的时候，会分别影响偏航角、横滚角及俯仰角。

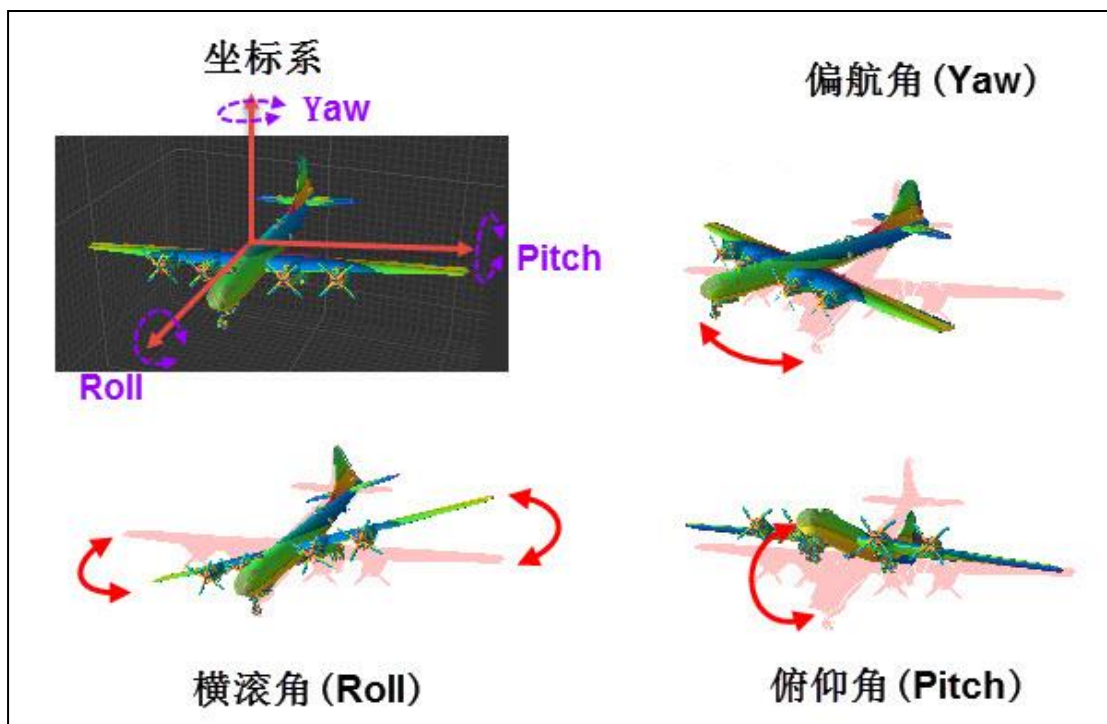


图 52-1 表示飞机姿态的偏航角、横滚角及俯仰角

假如我们知道飞机初始时是左上角的状态，只要想办法测量出基于原始状态的三个姿态角的变化量，再进行叠加，就可以获知它的实时姿态了。

2. 坐标系

抽象来说，姿态是“载体坐标系”与“地理坐标系”之间的转换关系。

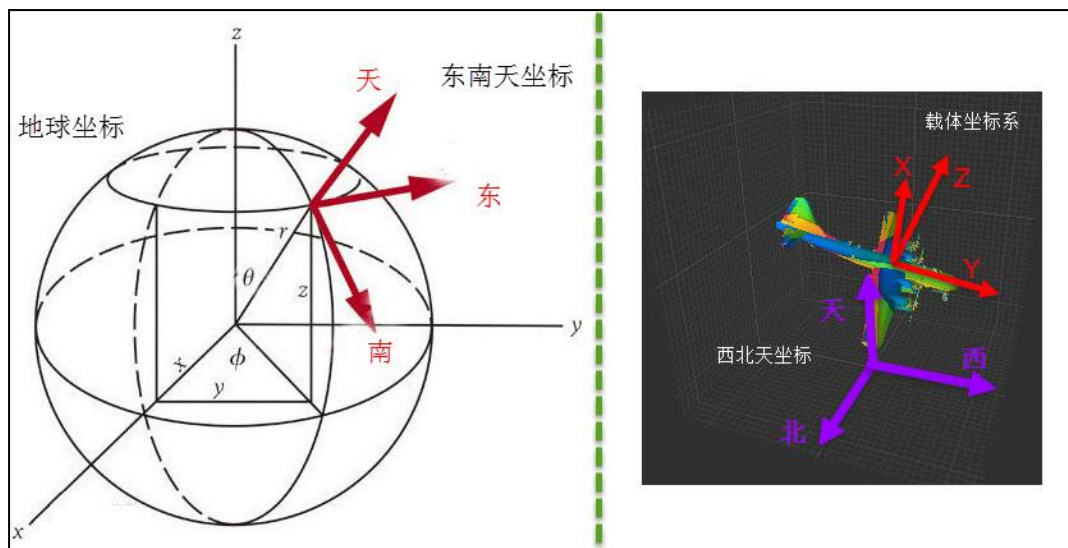


图 52-2 地球坐标系、地理坐标系与载体坐标系

我们先来了解三种常用的坐标系：

- ❑ 地球坐标系：以地球球心为原点，Z 轴沿地球自转轴方向，X、Y 轴在赤道平面内的坐标系。
- ❑ 地理坐标系：它的原点在地球表面(或运载体所在的点)，Z 轴沿当地地理垂线的方向(重力加速度方向)，XY 轴沿当地经纬线的切线方向。根据各个轴方向的不同，可选为“东北天”、“东南天”、“西北天”等坐标系。这是我们日常生活中使用的坐标系，平时说的东南西北方向与这个坐标系东南西北的概念一致。
- ❑ 载体坐标系：载体坐标系以运载体的质心为原点，一般根据运载体自身结构方向构成坐标系，如 Z 轴上由原点指向载体顶部，Y 轴指向载体头部，X 轴沿载体两侧方向。上面说基于飞机建立的坐标系就是一种载体坐标系，可类比到汽车、舰船、人体、动物或手机等各种物体。

地理坐标系与载体坐标系都以载体为原点，所以它们可以经过简单的旋转进行转换，载体的姿态角就是根据载体坐标系与地理坐标系的夹角来确定的。配合图 52-1，发挥您的空间想象力，假设初始状态中，飞机的 Z 轴、X 轴及 Y 轴分别与地理坐标系的天轴、北轴、东轴平行。如当飞机绕自身的“Z”轴旋转，它会使自身的“Y”轴方向与地理坐标系的“南北”方向偏离一定角度，该角度就称为偏航角(Yaw)；当载体绕自身的“X”轴旋转，它会使自身的“Z”轴方向与地理坐标系的“天地”方向偏离一定角度，该角度称为俯仰角(Pitch)；当载体绕自身的“Y”轴旋转，它会使自身的“X”轴方向与地理坐标系的“东西”方向偏离一定角度，该角度称为横滚角。

表 52-1 姿态角的关系

坐标系间的旋转角度	说明	载体自身旋转
偏航角(Yaw)	Y 轴与北轴的夹角	绕载体 Z 轴旋转可改变
俯仰角(Pitch)	Z 轴与天轴的夹角	绕载体 X 轴旋转可改变
横滚角(Roll)	X 轴与东轴的夹角	绕载体 Y 轴旋转可改变

这些角度也称欧拉角，是用于描述姿态的非常直观的角度。

52.1.2 利用陀螺仪检测角度

最直观的角度检测器就是陀螺仪了，见图 52-3，它可以检测物体绕坐标轴转动的“角速度”，如同将速度对时间积分可以求出路程一样，将角速度对时间积分就可以计算出旋转的“角度”。

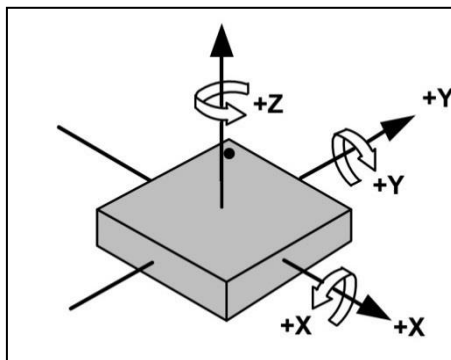


图 52-3 陀螺仪检测示意图

陀螺仪检测的缺陷

由于陀螺仪测量角度时使用积分，会存在积分误差，见图 52-4，若积分时间 Dt 越小，误差就越小。这十分容易理解，例如计算路程时，假设行车时间为 1 小时，我们随机选择行车过程某个时刻的速度 V_t 乘以 1 小时，求出的路程误差是极大的，因为行车的过程中并不是每个时刻都等于该时刻速度的，如果我们每 5 分钟检测一次车速，可得到 V_{t1} 、 V_{t2} 、 V_{t3} ... V_{t12} 这 12 个时刻的车速，对各个时刻的速度乘以时间间隔(5 分钟)，并对这 12 个结果求和，就可得出一个相对精确的行车路程了，不断提高采样频率，就可以使积分时间 Dt 变小，降低误差。

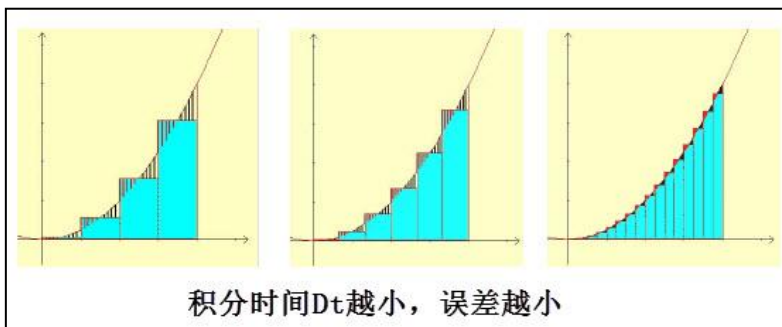


图 52-4 积分误差

同样地，提高陀螺仪传感器的采样频率，即可减少积分误差，目前非常普通的陀螺仪传感器的采样频率都可以达到 8KHz，已能满足大部分应用的精度要求。

更难以解决的是器件本身误差带来的问题。例如，某种陀螺仪的误差是 0.1 度/秒，当陀螺仪静止不动时，理想的角速度应为 0，无论它静止多久，对它进行积分测量得的旋转角度都是 0，这是理想的状态；而由于存在 0.1 度/秒的误差，当陀螺仪静止不动时，它采样得的角速度一直为 0.1 度/秒，若静止了 1 分钟，对它进行积分测量得的旋转角度为 6 度，若静止了 1 小时，陀螺仪进行积分测量得的旋转角度就是 360 度，即转过了一整圈，这就

变得无法忍受了。只有当正方向误差和负方向误差能正好互相抵消的时候，才能消除这种累计误差。

52.1.3 利用加速度计检测角度

由于直接用陀螺仪测量角度在长时间测量时会产生累计误差，因而我们又引入了检测倾角的传感器。



图 52-5 T 字型水平仪

测量倾角最常见的例子是建筑中使用的水平仪，在重力的影响下，水平仪内的气泡能大致反映水柱所在直线与重力方向的夹角关系，利用图 52-5 中的 T 字型水平仪，可以检测出图 52-1 中说明的横滚角与俯仰角，但是偏航角是无法以这样的方式检测的。

在电子设备中，一般使用加速度传感器来检测倾角，它通过检测器件在各个方向的形变情况而采样得到受力数据，根据 $F=ma$ 转换，传感器直接输出加速度数据，因而被称为加速度传感器。由于地球存在重力场，所以重力在任何时刻都会作用于传感器，当传感器静止的时候(实际上加速度为 0)，传感器会在该方向检测出加速度 g ，不能认为重力方向测出的加速度为 g ，就表示传感器在该方向作加速度为 g 的运动。

当传感器的姿态不同时，它在自身各个坐标轴检测到的重力加速度是不一样的，利用各方向的测量结果，根据力的分解原理，可求出各个坐标轴与重力之间的夹角，见图 52-6。

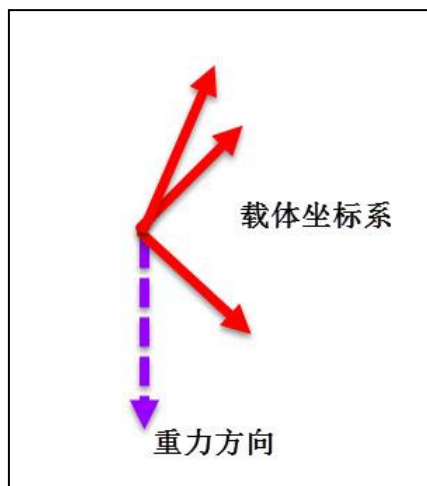


图 52-6 重力检测

因为重力方向是与地理坐标系的“天地”轴固连的，所以通过测量载体坐标系各轴与重力方向的夹角即可求得它与地理坐标系的角度旋转关系，从而获知载体姿态。

加速度传感器检测的缺陷

由于这种倾角检测方式是利用重力进行检测的，它无法检测到偏航角(Yaw)，原理跟 T 字型水平仪一样，无论如何设计水平仪，水泡都无法指示这样的角度。

另一个缺陷是加速度传感器并不会区分重力加速度与外力加速度，当物体运动的时候，它也会在运动的方向检测出加速度，特别在震动的状态下，传感器的数据会有非常大的数据变化，此时难以反应重力的实际值。

52.1.4 利用磁场检测角度

为了弥补加速度传感器无法检测偏航角(Yaw)的问题，我们再引入磁场检测传感器，它可以检测出各个方向上的磁场大小，通过检测地球磁场，它可实现指南针的功能，所以也被称为电子罗盘。由于地磁场与地理坐标系的“南北”轴固联，利用磁场检测传感器的指南针功能，就可以测量出偏航角(Yaw)了。

磁场检测器的缺陷

与指南针的缺陷一样，使用磁场传感器会受到外部磁场干扰，如载体本身的电磁场干扰，不同地理环境的磁铁矿干扰等等。

52.1.5 利用 GPS 检测角度

使用 GPS 可以直接检测出载体在地球上的坐标，假如载体在某时刻测得坐标为 A，另一时刻测得坐标为 B，利用两个坐标即可求出它的航向，即可以确定偏航角，且不受磁场的影响，但这种检测方式只有当载体产生大范围位移的时候才有效(GPS 民用精度大概为 10 米级)。

52.1.6 姿态融合与四元数

可以发现，使用陀螺仪检测角度时，在静止状态下存在缺陷，且受时间影响，而加速度传感器检测角度时，在运动状态下存在缺陷，且不受时间影响，刚好互补。假如我们同时使用这两种传感器，并设计一个滤波算法，当物体处于静止状态时，增大加速度数据的权重，当物体处于运动状态时，增大陀螺仪数据的权重，从而获得更准确的姿态数据。同理，检测偏航角，当载体在静止状态时，可增大磁场检测器数据的权重，当载体在运动状态时，增大陀螺仪和 GPS 检测数据的权重。这些采用多种传感器数据来检测姿态的处理算法被称为姿态融合。

在姿态融合解算的时候常常使用“四元数”来表示姿态，它由三个实数及一个虚数组成，因而被称之为四元数。使用四元数表示姿态并不直观，但因为使用欧拉角(即前面说的偏航角、横滚角及俯仰角)表示姿态的时候会有“万向节死锁”问题，且运算比较复杂，所以一般在数据处理的时候会使用四元数，处理完毕后再把四元数转换成欧拉角。在这里我们只要了解四元数是姿态的另一种表示方式即可，感兴趣的话可自行查阅相关资料。

52.2 传感器

1. 传感器工作原理

前文提到了各种传感器，在这里大致讲解一下传感器的工作原理。我们讲的传感器一般是指把物理量转化成电信号量的装置，见图 52-7。



图 52-7 传感器工作原理

敏感元件直接感受被测物理量，并输出与该物理量有确定关系的信号，经过转换元件将该物理量信号转换为电信号，变换电路对转换元件输出的电信号进行放大调制，最后输出容易检测的电信号量。例如，温度传感器可把温度量转化成电压信号量输出，且温度值与电压值成比例关系，我们只要使用 ADC 测量出电压值，并根据转换关系即可求得实际温度值。而前文提到的陀螺仪、加速度及磁场传感器也是类似的，它们检测的角速度、加速度及磁场强度与电压值有确定的转换关系。

2. 传感器参数

传感器一般使用精度、分辨率及采样频率这些参数来进行比较，衡量它的性能，见表 52-2。

表 52-2 传感器参数

参数	说明
线性误差	指传感器测量值与真实物理量值之间的拟合度误差。
分辨率	指传感器可检测到的最小物理量的单位。
采样频率	指在单位时间内的采样次数。

其中误差与分辨率是比较容易混淆的概念，以使用尺子测量长度为例，误差就是指尺子准不准，使用它测量出 10 厘米，与计量机构标准的 10 厘米有多大区别，若区别在 5 毫米以内，我们则称这把尺子的误差为 5 毫米。而分辨率是指尺子的最小刻度值，假如尺子的最小刻度值为 1 厘米，我们称这把尺子的分辨率为 1 厘米，它只能用于测量厘米级的尺寸，对于毫米级的长度，这就无法用这把尺子进行测量了。如果把尺子加热拉长，尺子的误差会大于 5 毫米，但它的分辨率仍为 1 厘米，只是它测出的 1 厘米值与真实值之间差得更远了。

3. 物理量的表示方法

大部分传感器的输出都是与电压成比例关系的，电压值一般采用 ADC 来测量，而 ADC 一般有固定的位数，如 8 位 ADC、12 位 ADC 等，ADC 的位数会影响测量的分辨率及量程。例如图 52-8，假设用一个 2 位的 ADC 来测量长度，2 位的 ADC 最多只能表示 0、1、2、3 这四个数，假如它的分辨率为 20 厘米，那么它最大的测量长度为 60 厘米，假如

它的分辨率为 10 厘米，那么它的最大测量长度为 30 厘米，由此可知，对于特定位数的 ADC，量程和分辨率不可兼得。

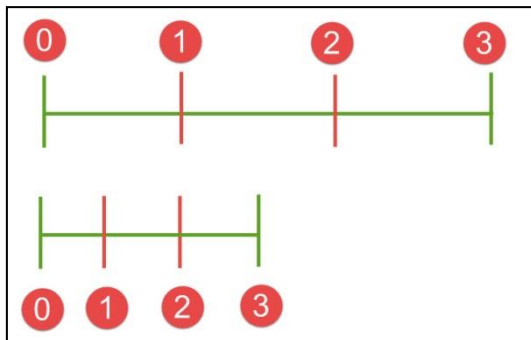


图 52-8 ADC 表示的物理量范围

在实际应用中，常常直接用 ADC 每位表征的物理量值来表示分辨率，如每位代表 20 厘米，我们称它的分辨率为 1LSB/20cm，它等效于 5 位表示 1 米：5LSB/m。其中的 LSB（Least Significant Bit），意为最 ADC 的低有效位。

使用采样得到的 ADC 数值，除以分辨率，即可求取得到物理量。例如使用分辨率为 5LSB/m、线性误差为 0.1m 的传感器进行长度测量，其 ADC 采样得到数据值为“20”，可计算知道该传感器的测量值为 4 米，而该长度的真实值介于 3.9-4.1 米之间。

52.3 MPU6050 简介

接下来我们使用传感器实例来讲解如何检测物体的姿态。在我们的 STM32F4 实验板上有一个 MPU6050 芯片，它是一种六轴传感器模块，采用 InvenSense 公司的 MPU6050 作为主芯片，能同时检测三轴加速度、三轴陀螺仪(三轴角速度)的运动数据以及温度数据。利用 MPU6050 芯片内部的 DMP 模块（Digital Motion Processor 数字运动处理器），可对传感器数据进行滤波、融合处理，它直接通过 I2C 接口向主控器输出姿态解算后的姿态数据，降低主控器的运算量。其姿态解算频率最高可达 200Hz，非常适合用于对姿态控制实时要求较高的领域。常见应用于手机、智能手环、四轴飞行器及计步器等姿态检测。

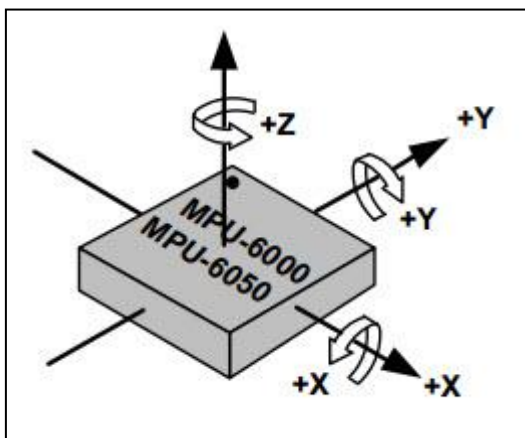


图 52-9 MPU6050 传感器的坐标及方向

图 52-9 中表示的坐标系及旋转符号标出了 MPU6050 传感器的 XYZ 轴的加速度有角速度的正方向。

52.4 MPU6050 的特性参数

实验板中使用的 MPU6050 传感器参数见表 52-3。

表 52-3 MPU6050 的特性参数

参数	说明
供电	3.3V-5V
通讯接口	I2C 协议，支持的 I2C 时钟最高频率为 400KHz
测量维度	加速度：3 维 陀螺仪：3 维
ADC 分辨率	加速度：16 位 陀螺仪：16 位
加速度测量范围	$\pm 2g$ 、 $\pm 4g$ 、 $\pm 8g$ 、 $\pm 16g$ 其中 g 为重力加速度常数， $g=9.8m/s^2$
加速度最高分辨率	16384 LSB/g
加速度线性误差	0.1g
加速度输出频率	最高 1000Hz
陀螺仪测量范围	$\pm 250^\circ/s$ 、 $\pm 500^\circ/s$ 、 $\pm 1000^\circ/s$ 、 $\pm 2000^\circ/s$ 、
陀螺仪最高分辨率	131 LSB/($^\circ/s$)
陀螺仪线性误差	0.1 $^\circ/s$
陀螺仪输出频率	最高 8000Hz
DMP 姿态解算频率	最高 200Hz
温度传感器测量范围	-40~ +85 $^\circ C$
温度传感器分辨率	340 LSB/ $^\circ C$
温度传感器线性误差	$\pm 1^\circ C$
工作温度	-40~ +85 $^\circ C$
功耗	500uA~3.9mA (工作电压 3.3V)

该表说明，加速度与陀螺仪传感器的 ADC 均为 16 位，它们的量程及分辨率可选多种模式，见图 52-11，量程越大，分辨率越低。

AFS_SEL	Full Scale Range	LSB Sensitivity
0	$\pm 2g$	16384 LSB/g
1	$\pm 4g$	8192 LSB/g
2	$\pm 8g$	4096 LSB/g
3	$\pm 16g$	2048 LSB/g

图 52-10 加速度配置跟量程的关系

FS_SEL selects the full scale range of the gyroscope outputs according to the following table.

FS_SEL	Full Scale Range
0	± 250 °/s
1	± 500 °/s
2	± 1000 °/s
3	± 2000 °/s

图 52-11 陀螺仪的几种量程配置

从表中还可了解到传感器的加速度及陀螺仪的采样频率分别为 1000Hz 及 8000Hz，它们是指加速度及角速度数据的采样频率，我们可以使用 STM32 控制器把这些数据读取出来然后进行姿态融合解算，以求出传感器当前的姿态(即求出偏航角、横滚角、俯仰角)。而如果我们使用传感器内部的 DMP 单元进行解算，它可以直接对采样得到的加速度及角速度进行姿态解算，解算得到的结果再输出给 STM32 控制器，即 STM32 无需自己计算，可直接获取偏航角、横滚角及俯仰角，该 DMP 每秒可输出 200 次姿态数据。

52.5 MPU6050—获取原始数据实验

这一小节我们学习如何使用 STM32 控制 MPU6050 传感器读取加速度、角速度及温度数据。在控制传感器时，使用到了 STM32 的 I2C 驱动，就如同控制 STM32 一样，对 MPU6050 传感器的不同寄存器写入不同内容可以实现不同模式的控制，从特定的寄存器读取内容则可获取测量数据，这部分关于 MPU6050 具体寄存器的内容我们不再展开，请您查阅《MPU-60X0 寄存器》手册获知。

52.5.1 硬件设计

STM32 与 MPU6050 的硬件连接见图 46-8。

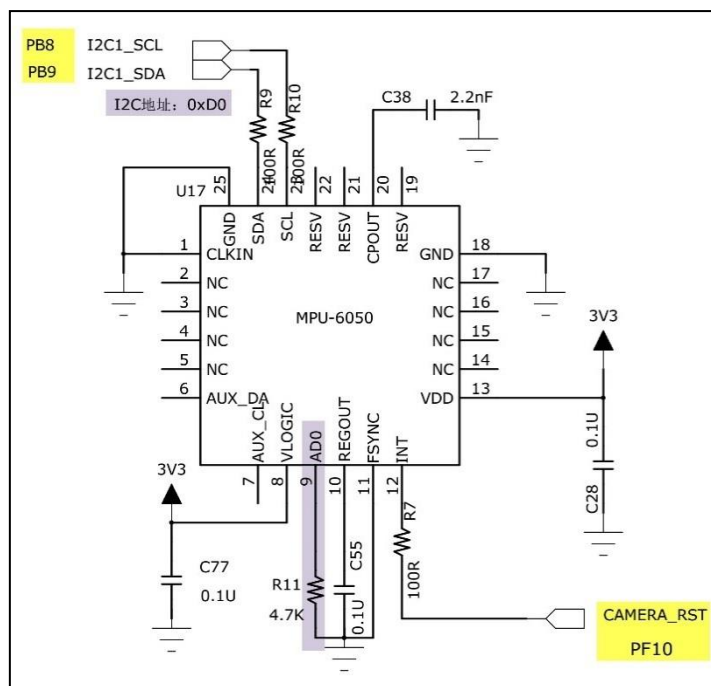


图 52-12 STM32 与 MPU6050 的硬件连接

它的硬件连接非常简单，SDA 与 SCL 引出到 STM32 的 I2C 引脚，注意图中的 I2C 没有画出上拉电阻，只是因为实验板中其它芯片也使用了同样的 I2C 总线，电阻画到了其它芯片的图里，没有出现在这个图中而已。传感器的 I2C 设备地址可通过 AD0 引脚的电平控制，当 AD0 接地时，设备地址为 0x68(七位地址)，当 AD0 接电源时，设备地址为 0x69(七位地址)。另外，传感器的 INT 引脚接到了 STM32 的普通 IO 口，当传感器有新数据的时候会通过 INT 引脚通知 STM32。

由于 MPU6050 检测时是基于自己中心坐标系的，所以在自己设计硬件时，您需要考虑它与所在设备的坐标系统的关系。

52.5.2 软件设计

本小节讲解的是“MPU6050 基本数据读取”实验，请打开配套的代码工程阅读理解。为了方便展示及移植，我们把 STM32 的 I2C 驱动相关的代码都编写到“i2c.c”及“i2c.h”文件中，与 MPU6050 传感器相关的代码都写到“mpu6050.c”及“mpu6050.h”文件中，这些文件是我们自己编写的，不属于标准库的内容，可根据您的喜好命名文件。

1. 程序设计要点

- (1) 初始化 STM32 的 I2C；
- (2) 使用 I2C 向 MPU6050 写入控制参数；
- (3) 定时读取加速度、角速度及温度数据。

2. 代码分析

I2C 的硬件定义

本实验中的 I2C 驱动与 MPU6050 驱动分开主要是考虑到扩展其它传感器时的通用性，如使用磁场传感器、气压传感器都可以使用同样一个 I2C 驱动，这个驱动只要给出针对不同传感器时的不同读写接口即可。关于 STM32 的 I2C 驱动原理请参考读写 EEPROM 的章节，本章讲解的 I2C 驱动主要针对接口封装讲解，细节不再赘述。本实验中的 I2C 硬件定义见代码清单 52-1。

代码清单 52-1 I2C 的硬件定义(i2c.h 文件)

```
1 /*引脚定义*/
2
3 #define SENSORS_I2C_SCL_GPIO_PORT      GPIOB
4 #define SENSORS_I2C_SCL_GPIO_CLK      RCC_AHB1Periph_GPIOB
5 #define SENSORS_I2C_SCL_GPIO_PIN      GPIO_Pin_8
6 #define SENSORS_I2C_SCL_GPIO_PINSOURCE GPIO_PinSource8
7
8 #define SENSORS_I2C_SDA_GPIO_PORT      GPIOB
9 #define SENSORS_I2C_SDA_GPIO_CLK      RCC_AHB1Periph_GPIOB
10 #define SENSORS_I2C_SDA_GPIO_PIN      GPIO_Pin_9
11 #define SENSORS_I2C_SDA_GPIO_PINSOURCE GPIO_PinSource9
12
13 #define SENSORS_I2C_AF                  GPIO_AF_I2C1
14
15 #define SENSORS_I2C                      I2C1
16 #define SENSORS_I2C_RCC_CLK             RCC_APB1Periph_I2C1
```

这些宏根据传感器使用的 I2C 硬件封装起来了。

初始化 I2C

接下来利用这些宏对 I2C 进行初始化，初始化过程与 I2C 读写 EEPROM 中的无异，见代码清单 52-2。

代码清单 52-2 初始化 I2C (i2c.c 文件)

```
1 /**
2  * @brief  初始化 I2C 总线，使用 I2C 前需要调用
3  * @param  无
4  * @retval 无
5  */
6 void I2cMaster_Init(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9     I2C_InitTypeDef I2C_InitStructure;
10
11     /* Enable I2Cx clock */
12     RCC_APB1PeriphClockCmd(SENSORS_I2C_RCC_CLK, ENABLE);
13
14     /* Enable I2C GPIO clock */
15     RCC_AHB1PeriphClockCmd(SENSORS_I2C_SCL_GPIO_CLK |
16                             SENSORS_I2C_SDA_GPIO_CLK, ENABLE);
17
18     /* Configure I2Cx pin: SCL -----*/
19     GPIO_InitStructure.GPIO_Pin = SENSORS_I2C_SCL_GPIO_PIN;
20     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
21     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
22     GPIO_InitStructure.GPIO_OType = GPIO_OType_OD;
23     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
24
25     /* Connect pins to Periph */
26     GPIO_PinAFConfig(SENSORS_I2C_SCL_GPIO_PORT, SENSORS_I2C_SCL_GPIO_PINSOURCE,
27                      SENSORS_I2C_AF);
28     GPIO_Init(SENSORS_I2C_SCL_GPIO_PORT, &GPIO_InitStructure);
29
30     /* Configure I2Cx pin: SDA -----*/
31     GPIO_InitStructure.GPIO_Pin = SENSORS_I2C_SDA_GPIO_PIN;
32
33     /* Connect pins to Periph */
34     GPIO_PinAFConfig(SENSORS_I2C_SDA_GPIO_PORT, SENSORS_I2C_SDA_GPIO_PINSOURCE,
35                      SENSORS_I2C_AF);
36     GPIO_Init(SENSORS_I2C_SDA_GPIO_PORT, &GPIO_InitStructure);
37
38     I2C_DeInit(SENSORS_I2C);
39     I2C_InitStructure.I2C_Mode = I2C_Mode_I2C;
40     I2C_InitStructure.I2C_DutyCycle = I2C_DutyCycle_2;
41     I2C_InitStructure.I2C_OwnAddress1 = I2C_OWN_ADDRESS;
42     I2C_InitStructure.I2C_Ack = I2C_Ack_Enable;
43     I2C_InitStructure.I2C_AcknowledgedAddress = I2C_AcknowledgedAddress_7bit;
44     I2C_InitStructure.I2C_ClockSpeed = I2C_SPEED;
45
46     /* Enable the I2C peripheral */
47     I2C_Cmd(SENSORS_I2C, ENABLE);
48
49     /* Initialize the I2C peripheral */
50     I2C_Init(SENSORS_I2C, &I2C_InitStructure);
51
52     return;
53 }
```

对读写函数的封装

初始化完成后就是编写 I2C 读写函数了，这部分跟 EERPOM 的一样，主要是调用 STM32 标准库函数读写数据寄存器及标志位，本实验的这部分被编写进 ST_Sensors_I2C_WriteRegister 及 ST_Sensors_I2C_ReadRegister 中了，在它们之上，再封装成了 Sensors_I2C_WriteRegister 及 Sensors_I2C_ReadRegister，见代码清单 52-3。

代码清单 52-3 对读写函数的封装(i2c.c 文件)

```
1
2 /**
3  * @brief 写寄存器(多次尝试)，这是提供给上层的接口
4  * @param slave_addr: 从机地址
5  * @param reg_addr: 寄存器地址
6  * @param len: 写入的长度
7  * @param data_ptr: 指向要写入的数据
8  * @retval 正常为 0，不正常为非 0
9  */
10 int Sensors_I2C_WriteRegister(unsigned char slave_addr,
11                               unsigned char reg_addr,
12                               unsigned short len,
13                               const unsigned char *data_ptr)
14 {
15     char retries=0;
16     int ret = 0;
17     unsigned short retry_in_msec = Get_I2C_Retry();
18
19 tryWriteAgain:
20     ret = 0;
21     ret = ST_Sensors_I2C_WriteRegister( slave_addr, reg_addr, len, data_ptr);
22
23     if (ret && retry_in_msec) {
24         if ( retries++ > 4 )
25             return ret;
26
27         Delay(retry_in_msec);
28         goto tryWriteAgain;
29     }
30     return ret;
31 }
32
33 /**
34  * @brief 读寄存器(多次尝试)，这是提供给上层的接口
35  * @param slave_addr: 从机地址
36  * @param reg_addr: 寄存器地址
37  * @param len: 要读取的长度
38  * @param data_ptr: 指向要存储数据的指针
39  * @retval 正常为 0，不正常为非 0
40  */
41 int Sensors_I2C_ReadRegister(unsigned char slave_addr,
42                              unsigned char reg_addr,
43                              unsigned short len,
44                              unsigned char *data_ptr)
45 {
46     char retries=0;
47     int ret = 0;
48     unsigned short retry_in_msec = Get_I2C_Retry();
49
50 tryReadAgain:
51     ret = 0;
52     ret = ST_Sensors_I2C_ReadRegister( slave_addr, reg_addr, len, data_ptr);
53
54     if (ret && retry_in_msec) {
```

```
55         if ( retries++ > 4 )
56             return ret;
57
58         Delay(retry_in_mlsec);
59         goto tryReadAgain;
60     }
61     return ret;
62 }
```

封装后的函数主要是增加了错误重试机制，若读写出现错误，则会进行多次尝试，多次尝试均失败后会返回错误代码。这个函数作为 I2C 驱动对外的接口，其它使用 I2C 的传感器调用这个函数进行读写寄存器。

MPU6050 的寄存器定义

MPU6050 有各种各样的寄存器用于控制工作模式，我们把这些寄存器的地址、寄存器位使用宏定义到了 mpu6050.h 文件中了，见代码清单 52-4。

代码清单 52-4 MPU6050 的寄存器定义(mpu6050.h)

```
1 // MPU6050, Standard address 0x68
2 #define MPU6050_ADDRESS      0x68
3 #define MPU6050_WHO_AM_I     0x75
4 #define MPU6050_SMPLRT_DIV   0 //8000Hz
5 #define MPU6050_DLPF_CFG     0
6 #define MPU6050_GYRO_OUT     0x43 //MPU6050 陀螺仪数据寄存器地址
7 #define MPU6050_ACC_OUT      0x3B //MPU6050 加速度数据寄存器地址
8
9 #define MPU6050_ADDRESS_AD0_LOW 0x68 //AD0 为低电平时的地址
10 #define MPU6050_ADDRESS_AD0_HIGH 0x69 //AD0 为高电平时的地址
11 #define MPU6050_DEFAULT_ADDRESS MPU6050_ADDRESS_AD0_LOW
12
13 #define MPU6050_RA_XG_OFFS_TC 0x00 //[7] PWR_MODE, [6:1] XG_OFFS_TC,
14                                     [0] OTP_BNK_VLD
15 #define MPU6050_RA_YG_OFFS_TC 0x01 //[7] PWR_MODE, [6:1] YG_OFFS_TC,
16                                     [0] OTP_BNK_VLD
17 #define MPU6050_RA_ZG_OFFS_TC 0x02 //[7] PWR_MODE, [6:1] ZG_OFFS_TC,
18                                     [0] OTP_BNK_VLD
19 #define MPU6050_RA_X_FINE_GAIN 0x03 //[7:0] X_FINE_GAIN
20 /*.....以下部分省略*/
```

初始化 MPU6050

根据 MPU6050 的寄存器功能定义，我们使用 I2C 往寄存器写入特定的控制参数，见代码清单 52-5。

代码清单 52-5 初始化 MPU6050

```
1
2 /**
3  * @brief   写数据到 MPU6050 寄存器
4  * @param   reg_add:寄存器地址
5  * @param   reg_data:要写入的数据
6  * @retval
7  */
8 void MPU6050_WriteReg(u8 reg_add,u8 reg_dat)
9 {
10     Sensors_I2C_WriteRegister(MPU6050_ADDRESS,reg_add,1,&reg_dat);
11 }
12
13 /**
14  * @brief   从 MPU6050 寄存器读取数据
15  * @param   reg_add:寄存器地址
```

```
16  * @param   Read: 存储数据的缓冲区
17  * @param   num: 要读取的数据量
18  * @retval
19  */
20 void MPU6050_ReadData(u8 reg_add,unsigned char* Read,u8 num)
21 {
22     Sensors_I2C_ReadRegister(MPU6050_ADDRESS,reg_add,num,Read);
23 }
24
25
26 /**
27  * @brief   初始化 MPU6050 芯片
28  * @param
29  * @retval
30  */
31 void MPU6050_Init(void)
32 {
33     int i=0,j=0;
34     //在初始化之前要延时一段时间，若没有延时，则断电后再上电数据可能会出错
35     for (i=0; i<1000; i++) {
36         for (j=0; j<1000; j++) {
37             ;
38         }
39     }
40     //解除休眠状态
41     MPU6050_WriteReg(MPU6050_RA_PWR_MGMT_1, 0x00);
42     //陀螺仪采样率
43     MPU6050_WriteReg(MPU6050_RA_SMPLRT_DIV , 0x07);
44     MPU6050_WriteReg(MPU6050_RA_CONFIG , 0x06);
45     //配置加速度传感器工作在 16G 模式
46     MPU6050_WriteReg(MPU6050_RA_ACCEL_CONFIG , 0x01);
47     //陀螺仪自检及测量范围，典型值：0x18(不自检，2000deg/s)
48     MPU6050_WriteReg(MPU6050_RA_GYRO_CONFIG, 0x18);
49 }
```

这段代码首先使用 MPU6050_ReadData 及 MPU6050_WriteRed 函数封装了 I2C 的底层读写驱动，接下来用它们在 MPU6050_Init 函数中向 MPU6050 寄存器写入控制参数，设置了 MPU6050 的采样率、量程(分辨率)。

读传感器 ID

初始化后，可通过读取它的“WHO AM I”寄存器内容来检测硬件是否正常，该寄存器存储了 ID 号 0x68，见代码清单 52-6。

代码清单 52-6 读取传感器 ID

```
1
2 /**
3  * @brief   读取 MPU6050 的 ID
4  * @param
5  * @retval   正常返回 1，异常返回 0
6  */
7 uint8_t MPU6050ReadID(void)
8 {
9     unsigned char Re = 0;
10     MPU6050_ReadData(MPU6050_RA_WHO_AM_I,&Re,1);    //读器件地址
11     if (Re != 0x68) {
12         MPU_ERROR("检测不到 MPU6050 模块，请检查模块与开发板的接线");
13         return 0;
14     } else {
15         MPU_INFO("MPU6050 ID = %d\r\n",Re);
16         return 1;
17     }
18 }
```

读取原始数据

若传感器检测正常，就可以读取它数据寄存器获取采样数据了，见代码清单 52-7。

代码清单 52-7 读取传感器数据

```
1
2 /**
3  * @brief   读取 MPU6050 的加速度数据
4  * @param
5  * @retval
6  */
7 void MPU6050ReadAcc(short *accData)
8 {
9     u8 buf[6];
10    MPU6050_ReadData(MPU6050_ACC_OUT, buf, 6);
11    accData[0] = (buf[0] << 8) | buf[1];
12    accData[1] = (buf[2] << 8) | buf[3];
13    accData[2] = (buf[4] << 8) | buf[5];
14 }
15
16 /**
17  * @brief   读取 MPU6050 的角加速度数据
18  * @param
19  * @retval
20  */
21 void MPU6050ReadGyro(short *gyroData)
22 {
23     u8 buf[6];
24     MPU6050_ReadData(MPU6050_GYRO_OUT, buf, 6);
25     gyroData[0] = (buf[0] << 8) | buf[1];
26     gyroData[1] = (buf[2] << 8) | buf[3];
27     gyroData[2] = (buf[4] << 8) | buf[5];
28 }
29
30 /**
31  * @brief   读取 MPU6050 的原始温度数据
32  * @param
33  * @retval
34  */
35 void MPU6050ReadTemp(short *tempData)
36 {
37     u8 buf[2];
38     MPU6050_ReadData(MPU6050_RA_TEMP_OUT_H, buf, 2);    //读取温度值
39     *tempData = (buf[0] << 8) | buf[1];
40 }
41
42 /**
43  * @brief   读取 MPU6050 的温度数据，转化成摄氏度
44  * @param
45  * @retval
46  */
47 void MPU6050_ReturnTemp(float*Temperature)
48 {
49     short temp3;
50     u8 buf[2];
51
52     MPU6050_ReadData(MPU6050_RA_TEMP_OUT_H, buf, 2);    //读取温度值
53     temp3= (buf[0] << 8) | buf[1];
54     *Temperature=((double) (temp3 /340.0))+36.53;
55 }
```

其中前以上三个函数分别用于读取三轴加速度、角速度及温度值，这些都是原始的 ADC 数值(16 位长)，对于加速度和角速度，把读取得的 ADC 值除以分辨率，即可求得实

际物理量数值。最后一个函数 MPU6050_ReturnTemp 展示了温度 ADC 值与实际温度值间的转换，它是根据 MPU6050 的说明给出的转换公式进行换算的，注意陀螺仪检测的温度会受自身芯片发热的影响，严格来说它测量的是自身芯片的温度，所以用它来测量气温是不太准确的。对于加速度和角速度值我们没有进行转换，在下一小节中我们直接利用这些数据交给 DMP 单元，求解出姿态角。

main 函数

最后我们来看看本实验的 main 函数，见代码清单 52-8。

代码清单 52-8 main 函数

```
1
2 /*简单任务管理*/
3 uint32_t Task_Delay[NumOfTask]= {0};
4
5 /**
6  * @brief 主函数
7  * @param 无
8  * @retval 无
9  */
10 int main(void)
11 {
12     short Acel[3];
13     short Gyro[3];
14     float Temp;
15
16     SysTick_Init();
17     LED_GPIO_Config();
18     /*初始化 USART1*/
19     Debug_USART_Config();
20
21     //初始化 I2C
22     I2cMaster_Init();
23     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
24     printf("\r\n 这是一个 I2C 外设陀螺仪测试例程 \r\n");
25     //MPU6050 初始化
26     MPU6050_Init();
27
28     //检测 MPU6050
29     if (MPU6050ReadID() == 1) {
30         while (1) {
31             if (Task_Delay[0]==TASK_ENABLE) {
32                 LED2_TOGGLE;
33                 Task_Delay[0]=1000;
34             }
35             if (Task_Delay[1]==0) {
36                 MPU6050ReadAcc(Acel);
37                 printf("加速度: %d%d%d",Acel[0],Acel[1],Acel[2]);
38                 MPU6050ReadGyro(Gyro);
39                 printf("陀螺仪%d%d%d",Gyro[0],Gyro[1],Gyro[2]);
40                 MPU6050_ReturnTemp(&Temp);
41                 printf("温度%.2f\r\n",Temp);
42                 Task_Delay[1]=500; //更新一次数据，可根据自己的需求，提高采样频率，如 100ms 采样一次
43             }
44
45             //***** 下面是增加任务的格式*****//
46             // if(Task_Delay[i]==0)
47             // {
48             //     Task(i);
49             //     Task_Delay[i]=;
50             // }
```

```
51     }  
52     } else {  
53         printf("\r\n 没有检测到 MPU6050 传感器! \r\n");  
54         LED_RED;  
55         while (1);  
56     }  
57 }
```

本实验中控制 MPU6050 并没有使用中断检测，我们是利用 SysTick 定时器进行计时，隔一段时间读取 MPU6050 的数据寄存器获取采样数据的，代码中使用 Task_Delay 变量来控制定时时间，在 SysTick 中断里会每隔 1ms 对该变量值减 1，所以当它的值为 0 时表示定时时间到。

在 main 函数里，调用 I2cMaster_Init、MPU6050_Init 及 MPU6050ReadID 函数后，就在 while 循环里判断定时时间，定时时间到后就读取加速度、角速度及温度值，并使用串口打印信息到电脑端。

52.5.3 下载验证

用 USB 线连接开发板“USB TO UART”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板。在串口调试助手可看到 MPU6050 采样得到的调试信息。

52.6 MPU6050—利用 DMP 进行姿态解算

上一小节我们仅利用 MPU6050 采集了原始的数据，如果您对姿态解算的算法深有研究，可以自行编写姿态解算的算法，并利用这些数据，使用 STM32 进行姿态解算，解算后输出姿态角。而由于 MPU6050 内部集成了 DMP，不需要 STM32 参与解算，可直接输出姿态角，也不需要对接算算法作深入研究，非常方便，本章讲解如何使用 DMP 进行解算。

实验中使用的代码主体是从 MPU6050 官方提供的驱动《motion_driver_6.12》移植过来的，该资料包里提供了基于 STM32F4 控制器的源代码及使用 python 语言编写的上位机，资料中还附带了说明文档，请您充分利用官方自带的资料学习。

52.6.1 硬件设计

硬件设计与上一小节实验中的完全一样，且软件中使用了 INT 引脚产生的中断信号。

52.6.2 软件设计

本小节讲解的是“MPU6050_python 上位机”实验，请打开配套的代码工程阅读理解。本工程是从官方代码移植过来的(IAR 工程移植至 MDK)，改动并不多，我们主要给读者讲解一下该驱动的设计思路，方便应用。由于本工程的代码十分庞大，在讲解到某些函数时，请善用 MDK 的搜索功能，从而在工程中查找出对应的代码。

1. 程序设计要点

- (1) 提供 I2C 读写接口、定时服务及 INT 中断处理；
- (2) 从陀螺仪中获取原始数据并处理；
- (3) 更新数据并输出。

2. 代码分析

官方的驱动主要是为了 MPL 软件库(Motion Processing Library)，要移植该软件库我们需要为它提供 I2C 读写接口、定时服务以及 MPU6050 的数据更新标志。若需要输出调试信息到上位机，还需要提供串口接口。

I2C 读写接口

MPL 库的内部对 I2C 读写时都使用 i2c_write 及 i2c_read 函数，在文件 “inv_mpu.c” 中给出了它们的接口格式，见代码清单 52-1。

代码清单 52-9 I2C 读写接口(inv_mpu.c 文件)

```
1 /* The following functions must be defined for this platform:
2  * i2c_write(unsigned char slave_addr, unsigned char reg_addr,
3  *           unsigned char length, unsigned char const *data)
4  * i2c_read(unsigned char slave_addr, unsigned char reg_addr,
5  *           unsigned char length, unsigned char *data)
6  */
7
8 #define i2c_write    Sensors_I2C_WriteRegister
9 #define i2c_read     Sensors_I2C_ReadRegister
```

这些接口的格式与我们上一小节写的 I2C 读写函数 Sensors_I2C_ReadRegister 及 Sensors_I2C_WriteRegister 一致，所以可直接使用宏替换。

提供定时服务

MPL 软件库中使用到了延时及时间戳功能，要求需要提供 delay_ms 函数实现毫秒级延时，提供 get_ms 获取毫秒级的时间戳，它们的接口格式也在 “inv_mpu.c” 文件中给出，见代码清单 52-2。

代码清单 52-10 定时服务接口 (inv_mpu.c 文件)

```
1 /*
2  * delay_ms(unsigned long num_ms)
3  * get_ms(unsigned long *count)
4  */
5
6 #define delay_ms    Delay_ms
7 #define get_ms      get_tick_count
```

我们为接口提供的 Delay_ms 及 get_tick_count 函数定义在 bsp_SysTick.c 文件，我们使用 SysTick 每毫秒产生一次中断，进行计时，见代码清单 52-11。

代码清单 52-11 使用 SysTick 进行定时 (bsp_SysTick.c)

```
1
2 static __IO u32 TimingDelay;
3 static __IO uint32_t g_ul_ms_ticks=0;
4 /**
5  * @brief    us 延时程序,1ms 为一个单位
6  * @param
7  * @arg nTime: Delay_ms( 1 ) 则实现的延时为 1 ms
8  * @retval 无
9  */
10 void Delay_ms(__IO u32 nTime)
11 {
12     TimingDelay = nTime;
13
14     while (TimingDelay != 0);
```

```
15 }
16
17 /**
18  * @brief 获取节拍程序
19  * @param 无
20  * @retval 无
21  * @attention 在 SysTick 中断函数 SysTick_Handler() 调用
22  */
23 void TimingDelay_Decrement(void)
24 {
25     if (TimingDelay != 0x00) {
26         TimingDelay--;
27     }
28 }
29
30 /**
31  * @brief 获取当前毫秒值
32  * @param 存储最新毫秒值的变量
33  * @retval 无
34  */
35 int get_tick_count(unsigned long *count)
36 {
37     count[0] = g_ul_ms_ticks;
38     return 0;
39 }
40
41 /**
42  * @brief 毫秒累加器，在中断里每毫秒加 1
43  * @param 无
44  * @retval 无
45  */
46 void TimeStamp_Increment (void)
47 {
48     g_ul_ms_ticks++;
49 }
```

上述代码中的 TimingDelay_Decrement 和 TimeStamp_Increment 函数是在 SysTick 的中断服务函数中被调用的，见代码清单 52-12。systick 被配置为每毫秒产生一次中断，而每次中断中会对 TimingDelay 变量减 1，对 g_ul_ms_ticks 变量加 1。它们分别用于 Delay_ms 函数利用 TimingDelay 的值进行阻塞延迟，而 get_tick_count 函数获取的时间戳即 g_ul_ms_ticks 的值。

代码清单 52-12 SysTick 的中断服务函数

```
1 /**
2  * @brief This function handles SysTick Handler.
3  * @param None
4  * @retval None
5  */
6 void SysTick_Handler(void)
7 {
8     TimingDelay_Decrement();
9     TimeStamp_Increment();
10 }
```

提供串口调试接口

MPL 代码库的调试信息输出函数都集中到了 log_stm32.c 文件中，我们可以为这些函数提供串口输出接口，以便把这些信息输出到上位机，见代码清单 52-3。

代码清单 52-13 串口调试接口(log_stm32.c 文件)

```
1 /*串口输出接口*/
```

```
2 int fputc(int ch)
3 {
4     /* 发送一个字节数据到 USART1 */
5     USART_SendData(DEBUG_USART, (uint8_t) ch);
6
7     /* 等待发送完毕 */
8     while (USART_GetFlagStatus(DEBUG_USART, USART_FLAG_TXE) == RESET);
9
10    return (ch);
11 }
12
13 /*输出四元数数据*/
14 void eMPL_send_quat(long *quat)
15 {
16     char out[PACKET_LENGTH];
17     int i;
18     if (!quat)
19         return;
20     memset(out, 0, PACKET_LENGTH);
21     out[0] = '$';
22     out[1] = PACKET_QUAT;
23     out[3] = (char)(quat[0] >> 24);
24     out[4] = (char)(quat[0] >> 16);
25     out[5] = (char)(quat[0] >> 8);
26     out[6] = (char)quat[0];
27     out[7] = (char)(quat[1] >> 24);
28     out[8] = (char)(quat[1] >> 16);
29     out[9] = (char)(quat[1] >> 8);
30     out[10] = (char)quat[1];
31     out[11] = (char)(quat[2] >> 24);
32     out[12] = (char)(quat[2] >> 16);
33     out[13] = (char)(quat[2] >> 8);
34     out[14] = (char)quat[2];
35     out[15] = (char)(quat[3] >> 24);
36     out[16] = (char)(quat[3] >> 16);
37     out[17] = (char)(quat[3] >> 8);
38     out[18] = (char)quat[3];
39     out[21] = '\r';
40     out[22] = '\n';
41
42     for (i=0; i<PACKET_LENGTH; i++) {
43         fputc(out[i]);
44     }
45 }
```

上述代码中的 fputc 函数是我们自己编写的串口输出接口，它与我们重定向 printf 函数定义的 fputc 函数很功能类似。下面的 eMPL_send_quat 函数是 MPL 库中的原函数，它用于打印“四元数信息”，在这个 log_stm32.c 文件中还有输出日志信息的 _MLPrintLog 函数，输出原始信息到专用上位机的 eMPL_send_data 函数，它们都调用了 fputc 进行输出。

MPU6050 的中断接口

与我们上一小节中的基础实验不同，为了高效处理采样数据，MPL 代码库使用了 MPU6050 的 INT 中断信号，为此我们要给提供中断接口，见代码清单 52-4。

代码清单 52-14 中断接口(stm32f4xx_it.c 文件)

```
1
2 #define MPU_IRQHandler          EXTI15_10_IRQHandler
3
4 void MPU_IRQHandler(void)
5 {
6
7     if (EXTI_GetITStatus(MPU_INT_EXTI_LINE) != RESET) { //确保是否产生了 EXTI Line 中断
8         /* Handle new gyro*/
9     }
```

```
9         gyro_data_ready_cb();
10
11         EXTI_ClearITPendingBit(MPU_INT_EXTI_LINE);    //清除中断标志位
12     }
13 }
```

在工程中我们把 MPU6050 与 STM32 相连的引脚配置成了中断模式，上述代码是该引脚的中断服务函数，在中断里调用了 MPL 代码库的 gyro_data_ready_cb 函数，它设置了标志变量 hal.new_gyro，以通知 MPL 库有新的数据，其函数定义见代码清单 52-15。

代码清单 52-15 设置标志变量(main.c 文件)

```
1 /* Every time new gyro data is available, this function is called in an
2  * ISR context. In this example, it sets a flag protecting the FIFO read
3  * function.
4  */
5 void gyro_data_ready_cb(void)
6 {
7     hal.new_gyro = 1;
8 }
```

main 函数执行流程

了解 MPL 移植需要提供的接口后，我们直接看 main 函数了解如何利用 MPL 库获取姿态数据，见代码清单 52-5。

代码清单 52-16 使用 MPL 进行姿态解算的过程

```
1 /**
2  * @brief main entry point.
3  * @par Parameters None
4  * @retval void None
5  * @par Required preconditions: None
6  */
7
8 int main(void)
9 {
10     inv_error_t result;
11     unsigned char accel_fsr, new_temp = 0;
12     unsigned short gyro_rate, gyro_fsr;
13     unsigned long timestamp;
14     struct int_param_s int_param;
15
16     SysTick_Init();
17     LED_GPIO_Config();
18     /*初始化 USART1*/
19     Debug_USART_Config();
20     EXTI_MPU_Config();
21     // Configure I2C
22     I2cMaster_Init();
23
24     result = mpu_init(&int_param);
25     if (result) {
26         MPL_LOGE("Could not initialize gyro.\n");
27         LED_RED;
28     } else {
29         LED_GREEN;
30     }
31
32     result = inv_init_mpl();
33     if (result) {
34         MPL_LOGE("Could not initialize MPL.\n");
35     }
36
37     /* Compute 6-axis and 9-axis quaternions. */
38     inv_enable_quaternion();
```

```
39     inv_enable_9x_sensor_fusion();
40
41     /* Update gyro biases when not in motion.
42      * WARNING: These algorithms are mutually exclusive.
43      */
44     inv_enable_fast_nomot();
45     /* inv_enable_motion_no_motion(); */
46     /* inv_set_no_motion_time(1000); */
47
48     /* Update gyro biases when temperature changes. */
49     inv_enable_gyro_tc();
50
51     /* Allows use of the MPL APIs in read_from_mpl. */
52     inv_enable_eMPL_outputs();
53
54     result = inv_start_mpl();
55     if (result == INV_ERROR_NOT_AUTHORIZED) {
56         while (1) {
57             MPL_LOGE("Not authorized.\n");
58         }
59     }
60     if (result) {
61         MPL_LOGE("Could not start the MPL.\n");
62     }
63
64     /* Get/set hardware configuration. Start gyro. */
65     /* Wake up all sensors. */
66
67     mpu_set_sensors(INV_XYZ_GYRO | INV_XYZ_ACCEL);
68     /* Push both gyro and accel data into the FIFO. */
69     mpu_configure_fifo(INV_XYZ_GYRO | INV_XYZ_ACCEL);
70     mpu_set_sample_rate(DEFAULT_MPU_HZ);
71
72     /* Read back configuration in case it was set improperly. */
73     mpu_get_sample_rate(&gyro_rate);
74     mpu_get_gyro_fsr(&gyro_fsr);
75     mpu_get_accel_fsr(&accel_fsr);
76
77     /* Sync driver configuration with MPL. */
78     /* Sample rate expected in microseconds. */
79     inv_set_gyro_sample_rate(1000000L / gyro_rate);
80     inv_set_accel_sample_rate(1000000L / gyro_rate);
81
82     /* Set chip-to-body orientation matrix.
83      * Set hardware units to dps/g's/degrees scaling factor.
84      */
85     inv_set_gyro_orientation_and_scale(
86         inv_orientation_matrix_to_scalar(gyro_pdata.orientation),
87         (long)gyro_fsr<<15);
88     inv_set_accel_orientation_and_scale(
89         inv_orientation_matrix_to_scalar(gyro_pdata.orientation),
90         (long)accel_fsr<<15);
91
92     /* Initialize HAL state variables. */
93
94     hal.sensors = ACCEL_ON | GYRO_ON;
95     hal.dmp_on = 0;
96     hal.report = 0;
97     hal.rx.cmd = 0;
98     hal.next_pedo_ms = 0;
99     hal.next_compass_ms = 0;
100    hal.next_temp_ms = 0;
101
102    /* Compass reads are handled by scheduler. */
103    get_tick_count(&timestamp);
104
105    /* To initialize the DMP:
106     * 1. Call dmp_load_motion_driver_firmware(). This pushes the DMP image in
```

```

107      *   inv_mpu_dmp_motion_driver.h into the MPU memory.
108      * 2. Push the gyro and accel orientation matrix to the DMP.
109      * 3. Register gesture callbacks. Don't worry, these callbacks won't be
110      *   executed unless the corresponding feature is enabled.
111      * 4. Call dmp_enable_feature(mask) to enable different features.
112      * 5. Call dmp_set_fifo_rate(freq) to select a DMP output rate.
113      * 6. Call any feature-specific control functions.
114      *
115      * To enable the DMP, just call mpu set dmp state(1). This function can
116      * be called repeatedly to enable and disable the DMP at runtime.
117      *
118      * The following is a short summary of the features supported in the DMP
119      * image provided in inv_mpu_dmp_motion_driver.c:
120      * DMP FEATURE LP QUAT: Generate a gyro-only quaternion on the DMP at
121      * 200Hz. Integrating the gyro data at higher rates reduces numerical
122      * errors (compared to integration on the MCU at a lower sampling rate).
123      * DMP FEATURE_6X_LP_QUAT: Generate a gyro/accel quaternion on the DMP at
124      * 200Hz. Cannot be used in combination with DMP_FEATURE_LP_QUAT.
125      * DMP FEATURE TAP: Detect taps along the X, Y, and Z axes.
126      * DMP FEATURE ANDROID ORIENT: Google's screen rotation algorithm. Triggers
127      * an event at the four orientations where the screen should rotate.
128      * DMP FEATURE_GYRO_CAL: Calibrates the gyro data after eight seconds of
129      * no motion.
130      * DMP FEATURE SEND RAW ACCEL: Add raw accelerometer data to the FIFO.
131      * DMP FEATURE SEND RAW GYRO: Add raw gyro data to the FIFO.
132      * DMP FEATURE SEND CAL GYRO: Add calibrated gyro data to the FIFO. Cannot
133      * be used in combination with DMP_FEATURE_SEND_RAW_GYRO.
134      */
135      dmp_load_motion_driver_firmware();
136      dmp_set_orientation(inv_orientation_matrix_to_scalar(gyro_pdata.orientation));
137      dmp_register_tap_cb(tap_cb);
138      dmp_register_android_orient_cb(android_orient_cb);
139      /*
140      * Known Bug -
141      * DMP when enabled will sample sensor data at 200Hz and output to FIFO at the rate
142      * specified in the dmp_set_fifo_rate API. The DMP will then sent an interrupt once
143      * a sample has been put into the FIFO. Therefore if the dmp_set_fifo_rate is at 25Hz
144      * there will be a 25Hz interrupt from the MPU device.
145      *
146      * There is a known issue in which if you do not enable DMP FEATURE TAP
147      * then the interrupts will be at 200Hz even if fifo rate
148      * is set at a different rate. To avoid this issue include the DMP_FEATURE_TAP
149      *
150      * DMP sensor fusion works only with gyro at +-2000dps and accel +-2G
151      */
152      hal.dmp_features = DMP_FEATURE_6X_LP_QUAT | DMP_FEATURE_TAP |
153                        DMP_FEATURE_ANDROID_ORIENT |
154                        DMP_FEATURE_SEND_RAW_ACCEL |
155                        DMP_FEATURE_SEND_CAL_GYRO | DMP_FEATURE_GYRO_CAL;
156      dmp_enable_feature(hal.dmp_features);
157      dmp_set_fifo_rate(DEFAULT_MPU_HZ);
158      mpu_set_dmp_state(1);
159      hal.dmp_on = 1;
160      while (1) {
161
162          unsigned long sensor_timestamp;
163          int new_data = 0;
164          if (USART_GetFlagStatus(DEBUG_USART, USART_FLAG_RXNE)) {
165              /* A byte has been received via USART. See handle_input for a list of
166               * valid commands.
167               */
168              USART_ClearFlag(DEBUG_USART, USART_FLAG_RXNE);
169              handle_input();
170          }
171          get_tick_count(&timestamp);
172
173          /* Temperature data doesn't need to be read with every gyro sample.
174           * Let's make them timer-based like the compass reads.
175           */
176          if (timestamp > hal.next_temp_ms) {
177              hal.next_temp_ms = timestamp + TEMP_READ_MS;

```

```
178         new_temp = 1;
179     }
180
181     if (hal.new_gyro && hal.dmp_on) {
182         short gyro[3], accel_short[3], sensors;
183         unsigned char more;
184         long accel[3], quat[4], temperature;
185         /* This function gets new data from the FIFO when the DMP is in
186          * use. The FIFO can contain any combination of gyro, accel,
187          * quaternion, and gesture data. The sensors parameter tells the
188          * caller which data fields were actually populated with new data.
189          * For example, if sensors == (INV_XYZ_GYRO | INV_WXYZ_QUAT), then
190          * the FIFO isn't being filled with accel data.
191          * The driver parses the gesture data to determine if a gesture
192          * event has occurred; on an event, the application will be notified
193          * via a callback (assuming that a callback function was properly
194          * registered). The more parameter is non-zero if there are
195          * leftover packets in the FIFO.
196          */
197         dmp_read_fifo(gyro, accel_short, quat, &sensor_timestamp, &sensors, &more);
198         if (!more)
199             hal.new_gyro = 0;
200         if (sensors & INV_XYZ_GYRO) {
201             /* Push the new data to the MPL. */
202             inv_build_gyro(gyro, sensor_timestamp);
203             new_data = 1;
204             if (new_temp) {
205                 new_temp = 0;
206                 /* Temperature only used for gyro temp comp. */
207                 mpu_get_temperature(&temperature, &sensor_timestamp);
208                 inv_build_temp(temperature, sensor_timestamp);
209             }
210         }
211         if (sensors & INV_XYZ_ACCEL) {
212             accel[0] = (long)accel_short[0];
213             accel[1] = (long)accel_short[1];
214             accel[2] = (long)accel_short[2];
215             inv_build_accel(accel, 0, sensor_timestamp);
216             new_data = 1;
217         }
218         if (sensors & INV_WXYZ_QUAT) {
219             inv_build_quat(quat, 0, sensor_timestamp);
220             new_data = 1;
221         }
222     }
223
224     if (new_data) {
225         inv_execute_on_data();
226         /* This function reads bias-compensated sensor data and sensor
227          * fusion outputs from the MPL. The outputs are formatted as seen
228          * in eMPL_outputs.c. This function only needs to be called at the
229          * rate requested by the host.
230          */
231         read_from_mpl();
232     }
233 }
234 }
```

如您所见，main 函数非常长，而且我们只是摘抄了部分，在原工程代码中还有很多代码，例如加入磁场数据使用 9 轴数据进行解算的功能(这是 MPU9150 的功能，MPU6050 不支持)以及其它工作模式相关的控制示例。上述 main 函数的主要执行流程概括如下：

- (1) 初始化 STM32 的硬件，如 SysTick、LED、调试串口、INT 中断引脚以及 I2C 外设的初始化；
- (2) 调用 MPL 库函数 mpu_init 初始化传感器的基本工作模式(以下过程调用的大部分都是 MPL 库函数，不再强调)；

- (3) 调用 `inv_init_mpl` 函数初始化 MPL 软件库，初始化后才能正常进行解算；
- (4) 设置各种运算参数，如四元数运算(`inv_enable_quaternion`)、6 轴或 9 轴数据融合(`inv_enable_9x_sensor_fusion`)等等；
- (5) 设置传感器的工作模式(`mpu_set_sensors`)、采样率(`mpu_set_sample_rate`)、分辨率(`inv_set_gyro_orientation_and_scale`)等等；
- (6) 当 STM32 驱动、MPL 库、传感器工作模式、DMP 工作模式等所有初始化工作都完成后进行 `while` 循环；
- (7) 在 `while` 循环中检测串口的输入，若串口有输入，则调用 `handle_input` 根据串口输入的字符(命令)，切换工作方式。这部分主要是为了支持上位机通过输入命令，根据进行不同的处理，如开、关加速度信息的采集或调试信息的输出等；
- (8) 在 `while` 循环中检测是否有数据更新(`if (hal.new_gyro && hal.dmp_on)`)，当有数据更新的时候产生 INT 中断，会使 `hal.new_gyro` 置 1 的，从而执行 `if` 里的条件代码；
- (9) 使用 `dmp_read_fifo` 把数据读取到 FIFO，这个 FIFO 是指 MPL 软件库定义的一个缓冲区，用来缓冲最新采集得的数据；
- (10) 调用 `inv_build_gyro`、`inv_build_temp`、`inv_build_accel` 及 `inv_build_quat` 函数处理数据角速度、温度、加速度及四元数数据，并对标志变量 `new_data` 置 1；
- (11) 在 `while` 循环中检测 `new_data` 标志位，当有新的数据时执行 `if` 里的条件代码；
- (12) 调用 `inv_execute_on_data` 函数更新所有数据及状态；
- (13) 调用 `read_from_mpl` 函数向输出最新的数据。

数据输出接口

在上面 `main` 中最后调用的 `read_from_mpl` 函数演示了如何调用 MPL 数据输出接口，通过这些接口我们可以获得想要的的数据，其函数定义见代码清单 52-17。

代码清单 52-17 MPL 的数据输出接口(main.c)

```
1  /* Get data from MPL.
2  * TODO: Add return values to the inv get sensor type xxx APIs to differentiate
3  * between new and stale data.
4  */
5  static void read_from_mpl(void)
6  {
7      long msg, data[9];
8      int8_t accuracy;
9      unsigned long timestamp;
10     float float_data[3] = {0};
11
12     if (inv_get_sensor_type_quat(data, &accuracy, (inv_time_t*)&timestamp)) {
13         /* Sends a quaternion packet to the PC. Since this is used by the Python
14          * test app to visually represent a 3D quaternion, it's sent each time
15          * the MPL has new data.
16          */
17         eMPL_send_quat(data);
18     }
19     /* Specific data packets can be sent or suppressed using USB commands. */
20     if (hal.report & PRINT_QUAT)
21         eMPL_send_data(PACKET_DATA_QUAT, data);
22 }
23
24 if (hal.report & PRINT_ACCEL) {
25     if (inv_get_sensor_type_accel(data, &accuracy,
```

```
26                                     (inv_time_t*)&timestamp))
27     eMPL_send_data(PACKET_DATA_ACCEL, data);
28 }
29 if (hal.report & PRINT_GYRO) {
30     if (inv_get_sensor_type_gyro(data, &accuracy,
31                                     (inv_time_t*)&timestamp))
32         eMPL_send_data(PACKET_DATA_GYRO, data);
33 }
34
35 if (hal.report & PRINT_EULER) {
36     if (inv_get_sensor_type_euler(data, &accuracy,
37                                     (inv_time_t*)&timestamp))
38         eMPL_send_data(PACKET_DATA_EULER, data);
39 }
40
41
42 /*****使用液晶屏显示数据*****/
43 if (1) {
44     char cStr [ 70 ];
45     unsigned long timestamp, step_count, walk_time;
46
47     /*获取欧拉角*/
48     if (inv_get_sensor_type_euler(data, &accuracy, (inv_time_t*)&timestamp)) {
49
50
51 #ifdef USE_LCD_DISPLAY
52     //inv_get_sensor_type_euler 读出的数据是 Q16 格式，所以左移 16 位.
53     sprintf ( cStr, "Pitch : %.4f ", data[0]*1.0/(1<<16) );
54     ILI9806G_DispStringLine_EN(LINE(7), cStr);
55
56     //inv_get_sensor_type_euler 读出的数据是 Q16 格式，所以左移 16 位.
57     sprintf ( cStr, "Roll  : %.4f ", data[1]*1.0/(1<<16) );
58     ILI9806G_DispStringLine_EN(LINE(8), cStr);
59     //inv_get_sensor_type_euler 读出的数据是 Q16 格式，所以左移 16 位.
60     sprintf ( cStr, "Yaw   : %.4f ", data[2]*1.0/(1<<16) );
61     ILI9806G_DispStringLine_EN (LINE(9), (uint8_t*) cStr);
62
63     /*温度*/
64     mpu_get_temperature(data, (inv_time_t*)&timestamp);
65     //inv_get_sensor_type_euler 读出的数据是 Q16 格式，所以左移 16 位.
66     sprintf ( cStr, "Temperature : %.2f ", data[0]*1.0/(1<<16) );
67     ILI9806G_DispStringLine_EN (LINE(10), (uint8_t*) cStr);
68 #endif
69
70     }
71
72     /*获取步数*/
73     get_tick_count(&timestamp);
74     if (timestamp > hal.next_pedo_ms) {
75
76         hal.next_pedo_ms = timestamp + PEDO_READ_MS;
77         dmp_get_pedometer_step_count(&step_count);
78         dmp_get_pedometer_walk_time(&walk_time);
79
80 #ifdef USE_LCD_DISPLAY
81         sprintf(cStr, "Walked steps : %ld steps over %ld milliseconds..", step_count, walk_time);
82         ILI9806G_DispStringLine_EN (LINE(11), (uint8_t*) cStr);
83 #endif
84     }
85 }
86 /*以下省略*/
87 }
88
```

上述代码展示了使用 `inv_get_sensor_type_quat`、`inv_get_sensor_type_accel`、`inv_get_sensor_type_gyro`、`inv_get_sensor_type_euler` 及 `dmp_get_pedometer_step_count` 函数分别获取四元数、加速度、角速度、欧拉角及计步器数据。

代码中的 `eMPL_send_data` 函数是使用串口按照 PYTHON 上位机格式进行提交数据，上位机根据这些数据对三维模型作相应的旋转。

另外我们自己在代码中加入了液晶显示的代码(`#ifdef USE_LCD_DISPLAY` 宏内的代码)，它把这些数据输出到实验板上的液晶屏上。

您可根据自己的数据使用需求，参考这个 `read_from_mpl` 函数对数据输出接口的调用方式，编写自己的应用。

52.6.3 下载验证

直接下载本程序到开发板，在液晶屏上会观察到姿态角、温度、计步器数据，改变开发板的姿态，数据会更新(计步器数据要模拟走路才会更新)，若直接连接串口调试助手，会接收到一系列的乱码信息，这是正常的，这些数据需要使用官方的 Python 上位机解码。

本实验适用于官方提供的 Python 上位机，它可以把采样的数据传送到上位机，上位机会显示三维模式的姿态。

注意：以下内容仅针对有 Python 编程语言基础的用户，若您不会 Python，而又希望观察到三维模型的姿态，请参考下一小节的实验，它的使用更为简单。

1. Python 上位机源代码及说明

MPU6050 官方提供的上位机的使用说明可在配套资料《motion_driver6.12》源码包 documentation 文件夹里的《Motion Driver 6.12 – Getting Started Guide》找到。上位机的源码在《motion_driver6.12》源码包的 eMPL-pythonclient 文件夹，里边有三个 python 文件，见图 52-13。

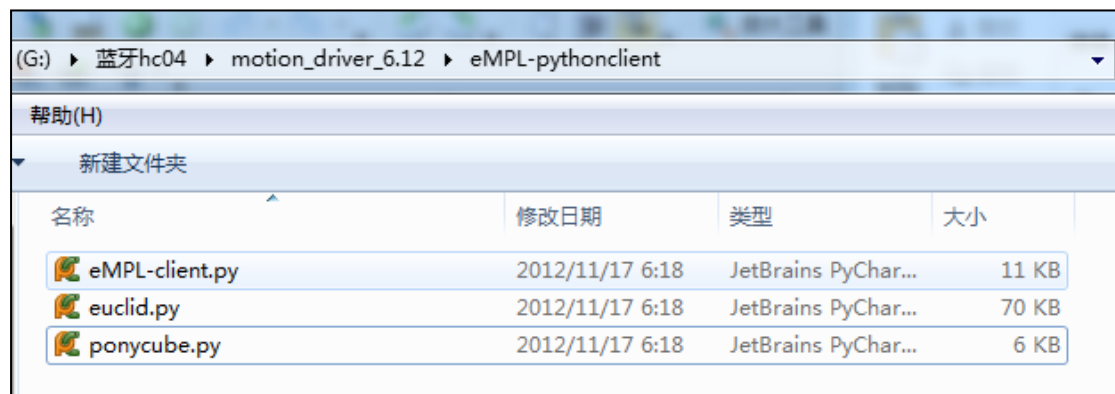


图 52-13 源码包里的 python 上位机源码

2. 安装 Python 环境

要利用上面的源码，需要先安装 Python 环境，该上位机支持 python2.7 环境(仅支持 32 位)，并且需要安装 Pyserial 库、Pygame 库。

可通过如下网址找到安装包。

Python: <https://www.python.org/downloads/>

Pyserial: <https://pypi.python.org/pypi/pyserial>

Pygame: <http://www.pygame.org/download.shtml>

3. Python 上位机的使用步骤

- ❑ 先把本 STM32 工程代码编译后下载到开发板上运行，确认开发板的 USB TO USART 接口已与电脑相连，正常时开发板的液晶屏现象跟上一章例程的现象一样。
- ❑ 使用命令行切换到 python 上位机的目录，执行如下命令：

```
python eMPL-client.py <COM PORT NUMBER>
```

其中<COM PORT NUMBER>参数是 STM32 开发板在电脑端的串口设备号，运行命令后会弹出一个 3D 图形窗口，显示陀螺仪的姿态，见图 52-14。(图中的“python2_32”是本机的 python2.7-32 位 python 命令的名字，用户默认用“python”命令即可。)

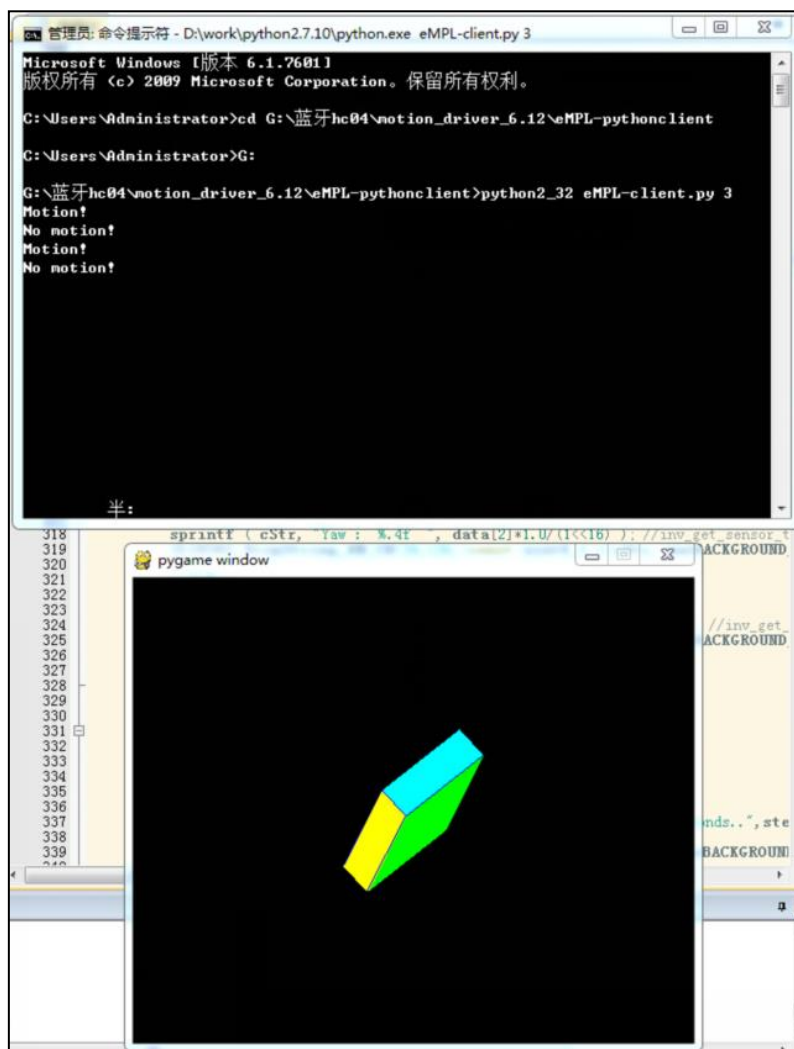


图 52-14 运行 python 上位机

- 这个上位机还可以接收命令来控制 STM32 进行数据输出，选中图中的 pygame window 窗口(弹出来的 3D 图形窗口)，然后按下键盘的字母 “a” 键，命令行窗口就会输出加速度信息，按下 “g” 键，就会输出陀螺仪信息。命令集说明如下：

- ‘8’ : Toggles Accel Sensor
- ‘9’ : Toggles Gyro Sensor
- ‘0’ : Toggles Compass Sensor
- ‘a’ : Prints Accel Data
- ‘g’ : Prints Gyro Data
- ‘c’ : Prints Compass Data
- ‘e’ : Prints Euler Data in radius
- ‘r’ : Prints Rotational Matrix Data
- ‘q’ : Prints Quaternions
- ‘h’ : Prints Heading Data in degrees
- ‘i’ : Prints Linear Acceleration data
- ‘o’ : Prints Gravity Vector data
- ‘w’ : Get compass accuracy and status
- ‘d’ : Register Dump
- ‘p’ : Turn on Low Power Accel Mode at 20Hz sampling
- ‘l’ : Load calibration data from flash memory
- ‘s’ : Save calibration data to flash memory
- ‘t’ : run factory self test and calibration routine
- ‘1’ : Change sensor output data rate to 10Hz
- ‘2’ : Change sensor output data rate to 20Hz
- ‘3’ : Change sensor output data rate to 40Hz
- ‘4’ : Change sensor output data rate to 50Hz
- ‘5’ : Change sensor output data rate to 100Hz
- ‘,’ : set interrupts to DMP gestures only
- ‘.’ : set interrupts to DMP data ready
- ‘6’ : Print Pedometer data
- ‘7’ : Reset Pedometer data
- ‘f’ : Toggle DMP on/off
- ‘m’ : Enter Low Power Interrupt Mode
- ‘x’ : Reset the MSP430
- ‘v’ : Toggle DMP Low Power Quaternion Generation

52.7 MPU6050—使用第三方上位机

上一小节中的实验必须配合使用官方提供的上位机才能看到三维模型，而且功能比较简单，所以在小节中我们演示如何把数据输出到第三方的上位机，直观地观察设备的姿态。

实验中我们使用的是“匿名飞控地面站 0512”版本的上位机，关于上位机的通讯协议可查阅《飞控通信协议》文档，或到他们的官方网站了解。

52.7.1 硬件设计

硬件设计与上一小节实验中的完全一样。

52.7.2 软件设计

本小节讲解的是“MPU6050_DMP 测试例程”实验，请打开配套的代码工程阅读理解。本小节的内容主体跟上一小节一样，区别主要是当获取得到数据后，本实验根据“匿名飞控”上位机的数据格式要求上传数据。

1. 程序设计要点

- (1) 了解上位机的通讯协议；
- (2) 根据协议格式上传数据到上位机；

2. 代码分析

通讯协议

要按照上位机的格式上传数据，首先要了解它的通讯协议，本实验中的上位机协议说明见表 52-4。

表 52-4 匿名上位机的通讯协议(部分)

帧	帧头	功能字	长度	数据	校验
STATUS	AAAA	01	LEN	int16 ROL*100 int16 PIT*100 int16 YAW*100 int32 ALT_USE u8 ARMED : A0 加锁 A1 解锁	SUM
SENDER	AAAA	02	LEN	int16 ACC_X int16 ACC_Y int16 ACC_Z int16 GYRO_X int16 GYRO_Y int16 GYRO_Z int16 MAG_X int16 MAG_Y int16 MAG_Z	SUM

表中说明了两种数据帧，分别是 STATUS 帧及 SENSER 帧，数据帧中包含帧头、功能字、长度、主体数据及校验和。“帧头”用于表示数据包的开始，均使用两个字节的 0xAA 表示；“功能字”用于区分数据帧的类型，0x01 表示 STATUS 帧，0x02 表示 SENSER 帧；“长度”表示后面主体数据内容的字节数；“校验和”用于校验，它是前面所有内容的和。

其中的 STATUS 帧用于向上位机传输横滚角、俯仰角及偏航角的值(100 倍)，SENSER 帧用于传输加速度、角速度及磁场强度的原始数据。

发送数据包

根据以上数据格式的要求，我们定义了两个函数，分别用于发送 STATUS 帧及 SENSER 帧，见代码清单 52-2。

代码清单 52-18 发送数据包（main.c 文件）

```
1
2 #define BYTE0(dwTemp)      (*(char *)(&dwTemp))
3 #define BYTE1(dwTemp)      (*(char *)(&dwTemp) + 1)
4 #define BYTE2(dwTemp)      (*(char *)(&dwTemp) + 2)
5 #define BYTE3(dwTemp)      (*(char *)(&dwTemp) + 3)
6
7 /**
8  * @brief 控制串口发送 1 个字符
9  * @param c:要发送的字符
10  * @retval none
11  */
12 void usart_send_char(uint8_t c)
13 {
14     //循环发送,直到发送完毕
15     while (USART_GetFlagStatus(DEBUG_USART, USART_FLAG_TXE) == RESET);
16     USART_SendData(DEBUG_USART, c);
17 }
18
19
20 /*函数功能：根据匿名最新上位机协议写的显示姿态的程序（上位机 0512 版本）
21  *具体协议说明请查看上位机软件的帮助说明。
22  */
23 void Data_Send_Status(float Pitch, float Roll, float Yaw)
24 {
25     unsigned char i=0;
26     unsigned char _cnt=0, sum = 0;
27     unsigned int _temp;
28     u8 data_to_send[50];
29
30     data_to_send[_cnt++] = 0xAA;
31     data_to_send[_cnt++] = 0xAA;
32     data_to_send[_cnt++] = 0x01;
33     data_to_send[_cnt++] = 0;
34
35     _temp = (int)(Roll*100);
36     data_to_send[_cnt++] = BYTE1(_temp);
37     data_to_send[_cnt++] = BYTE0(_temp);
38     _temp = 0 - (int)(Pitch*100);
39     data_to_send[_cnt++] = BYTE1(_temp);
40     data_to_send[_cnt++] = BYTE0(_temp);
41     _temp = (int)(Yaw*100);
42     data_to_send[_cnt++] = BYTE1(_temp);
43     data_to_send[_cnt++] = BYTE0(_temp);
44     _temp = 0;
45     data_to_send[_cnt++] = BYTE3(_temp);
46     data_to_send[_cnt++] = BYTE2(_temp);
```

```
47     data_to_send[_cnt++]=BYTE1(_temp);
48     data_to_send[_cnt++]=BYTE0(_temp);
49
50     data_to_send[_cnt++]=0xA0;
51
52     data_to_send[3] = _cnt-4;
53     //和校验
54     for (i=0; i<_cnt; i++)
55         sum+= data_to_send[i];
56     data_to_send[_cnt++]=sum;
57
58     //串口发送数据
59     for (i=0; i<_cnt; i++)
60         usart_send_char(data_to_send[i]);
61 }
62
63 /*函数功能：根据匿名最新上位机协议写的显示传感器数据（上位机 0512 版本）
64 *具体协议说明请查看上位机软件的帮助说明。
65 */
66 void Send_Data(int16_t *Gyro,int16_t *Accel)
67 {
68     unsigned char i=0;
69     unsigned char _cnt=0,sum = 0;
70     // unsigned int _temp;
71     u8 data_to_send[50];
72
73     data_to_send[_cnt++]=0xAA;
74     data_to_send[_cnt++]=0xAA;
75     data_to_send[_cnt++]=0x02;
76     data_to_send[_cnt++]=0;
77
78
79     data_to_send[_cnt++]=BYTE1(Accel[0]);
80     data_to_send[_cnt++]=BYTE0(Accel[0]);
81     data_to_send[_cnt++]=BYTE1(Accel[1]);
82     data_to_send[_cnt++]=BYTE0(Accel[1]);
83     data_to_send[_cnt++]=BYTE1(Accel[2]);
84     data_to_send[_cnt++]=BYTE0(Accel[2]);
85
86     data_to_send[_cnt++]=BYTE1(Gyro[0]);
87     data_to_send[_cnt++]=BYTE0(Gyro[0]);
88     data_to_send[_cnt++]=BYTE1(Gyro[1]);
89     data_to_send[_cnt++]=BYTE0(Gyro[1]);
90     data_to_send[_cnt++]=BYTE1(Gyro[2]);
91     data_to_send[_cnt++]=BYTE0(Gyro[2]);
92     data_to_send[_cnt++]=0;
93     data_to_send[_cnt++]=0;
94     data_to_send[_cnt++]=0;
95     data_to_send[_cnt++]=0;
96     data_to_send[_cnt++]=0;
97     data_to_send[_cnt++]=0;
98
99     data_to_send[3] = _cnt-4;
100    //和校验
101    for (i=0; i<_cnt; i++)
102        sum+= data_to_send[i];
103    data_to_send[_cnt++]=sum;
104
105    //串口发送数据
106    for (i=0; i<_cnt; i++)
107        usart_send_char(data_to_send[i]);
108 }
```

函数比较简单，就是根据输入的内容，一字节一字节地按格式封装好，然后调用串口发送到上位机。

发送数据

与上一小节一样，我们使用 `read_from_mpl` 函数输出数据，由于使用了不同的上位机，所以我们修改了它的具体内容，见代码清单 52-3。

代码清单 52-19 `read_from_mpl` 函数(main.c 文件)

```
1
2 extern struct inv_sensor_cal_t sensors;
3
4 /* Get data from MPL.
5  * TODO: Add return values to the inv get sensor type xxx APIs to differentiate
6  * between new and stale data.
7  */
8 static void read_from_mpl(void)
9 {
10     float Pitch,Roll,Yaw;
11     int8_t accuracy;
12     unsigned long timestamp;
13     long data[3];
14     /*获取欧拉角*/
15     inv_get_sensor_type_euler(data, &accuracy,(inv_time_t*)&timestamp);
16
17     //inv_get_sensor_type_euler 读出的数据是 Q16 格式，所以左移 16 位。
18     Pitch =data[0]*1.0/(1<<16) ;
19     Roll = data[1]*1.0/(1<<16);
20     Yaw = data[2]*1.0/(1<<16);
21
22     /*向匿名上位机发送姿态*/
23     Data_Send_Status(Pitch,Roll,Yaw);
24     /*向匿名上位机发送原始数据*/
25     Send_Data((int16_t *)&sensors.gyro.raw,(int16_t *)&sensors.accel.raw);
26 }
```

代码中调用 `inv_get_sensor_type_euler` 获取欧拉角，然后调用 `Data_Send_Status` 格式上传到上位机，而加速度及角速度的原始数据直接从 `sensors` 结构体变量即可获取，获取后调用 `Send_Data` 发送出去。

52.7.3 下载验证

直接下载本程序到开发板，在液晶屏上会观察到姿态角、温度、计步器数据，改变开发板的姿态，数据会更新(计步器数据要模拟走路才会更新)，若直接连接串口调试助手，会接收到一系列的乱码信息，这是正常的，这些数据需要使用“匿名飞控地面站”上位机解码。

若通过液晶屏的信息了解到 MPU6050 模块已正常工作，则可进一步在电脑上使用“**ANO_TC 匿名飞控地面站-0512.exe**”(以下简称“匿名上位机”)软件查看可视化数据。

实验步骤如下：

- (1) 确认开发板的 USB TO USART 接口已与电脑相连，确认电脑端能查看到该串口设备。
- (2) 打开配套资料里的“匿名上位机”软件，在软件界面打开 开发板对应的串口(波特率为 115200)，把“基本收码”、“高级收码”、“飞控波形”功能设置为 on 状态。点击上方图中的基本收发、波形显示、飞控状态图标，会弹出窗口。具体见下文软件配置图。

- (3) 在软件的“基本收发”、“波形显示”、“飞控状态”页面可看到滚动数据、随着模块晃动而变化的波形以及模块姿态的 3D 可视化图形。

